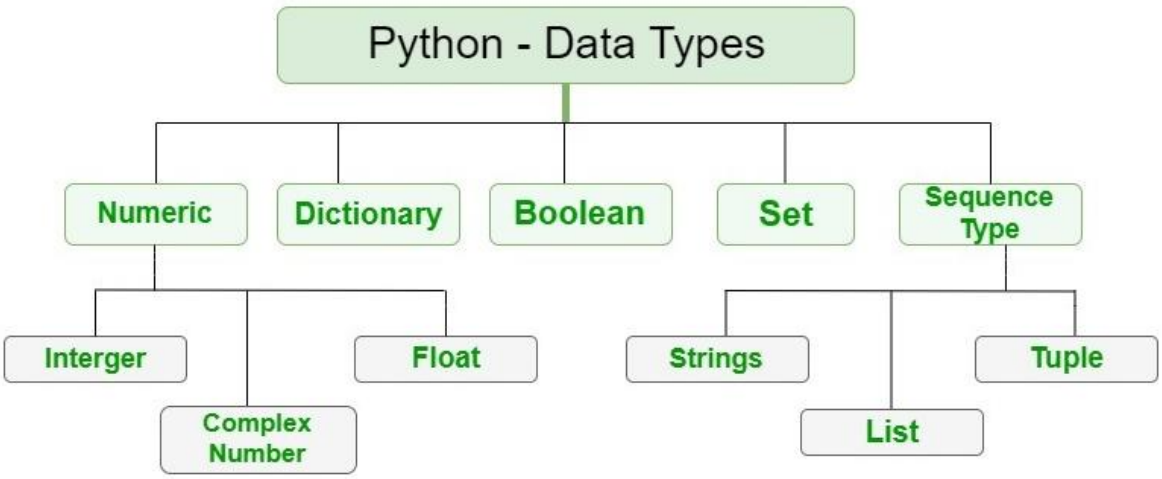
PYTHON

DAY-03

- Bitwise operators act on operands as if they were string of binary digits. The operate bit by bit. Formula for (~)= -(n+1).

Note: Boolean operators (`and`, `or`, `not`) are useful for decision-making and control flow in Python.



Python input and output

How to take input from the user in python

In Python, you can take input from the user using the input() function. Here's how it works:

1.Basic Input

Example code:

```
name = input("Enter your name: ")
print("Hello, " + name + "!")
```

input() reads input as a string.

The prompt inside input() helps guide the user.

2.Taking Integer Input

If you need an integer input, convert it using int().

Example code:

```
age = int(input("Enter your age: "))
print("You are", age, "years old.")
```

Taking Float Input

3.For decimal numbers, use float().

Example code:

```
price = float(input("Enter the price: "))
print("The price is", price)
```

4.Taking Multiple Inputs

You can take multiple inputs using split().

Example code:

```
x, y = input("Enter two numbers separated by space: ").split()
print("First number:", x, "Second number:", y)
```

Using map() to Convert Input

5.If multiple inputs need conversion:

```
x, y = map(int, input("Enter two numbers: ").split())
print("Sum:", x + y)
```

Implicit Type Conversion in Python

Introduction

Data types of variables are the kind of data a variable holds. Python has four basic data types, among others :

1. **Integer**
2. **Float**
3. **String**
4. **Boolean**

To know the type of data a variable holds, you can use the Python type() function.

```
>>> type('data type conversion')
<class 'str'>

>>> type(224)
<class 'int'>

>>> type(3.142)
<class 'float'>

>>> type(True)
<class 'bool'>

>>> type('False')
<class 'str'>

>>> type([1, 2, 3])
<class 'list'>

>>> type({'key': 'value'})
<class 'dict'>
```

Implicit Type Conversion in Python (Type Casting)

Implicit type conversion (also called Type Promotion) happens when Python automatically converts a lower data type to a higher data type to prevent data loss.

Examples of Implicit Type Conversion

Python performs this conversion automatically without user intervention.

Example 1: Integer to Float

```
num_int = 10   # Integer
num_float = num_int + 5.5  # Integer + Float → Float
print(num_float)  # Output: 15.5
print(type(num_float))  # Output: <class 'float'>
```

Explanation:

Python converts num_int (integer) to a float before performing addition.

Example 2: Integer to Complex

```
num_int = 10
num_complex = num_int + 2j  # Integer + Complex → Complex
print(num_complex)  # Output: (10+2j)
print(type(num_complex))  # Output: <class 'complex'>
```

Explanation:

Python converts num_int to a complex number before addition.

Example 3: Boolean to Integer

```
num = True + 5  # True is treated as 1
print(num)  # Output: 6
print(type(num))  # Output: <class 'int'>
```

Explanation:

True is internally treated as 1, so True + 5 = 6.

Example 4: Integer to String (No Implicit Conversion)

Python does not automatically convert an integer to a string in operations like concatenation:

```
num = 10
# print("The number is " + num)  # ✕ TypeError: can only concatenate str (not "int") to str

# Correct way (Explicit conversion)
print("The number is " + str(num))
```

Here, we need explicit conversion (str(num)) because Python does not implicitly convert an integer to a string.

When does Python NOT Perform Implicit Conversion?

Between incompatible types like string and integer.

When there is a risk of data loss (e.g., float to int).

Note:

Python automatically promotes smaller data types to larger ones (like int → float → complex), but when incompatible types are involved, explicit conversion (int(), float(), str(), etc.) is needed.

Explicit Type Conversion in Python

We transform an object's data type to the desired data type using explicit type conversion. To achieve explicit type conversion, we use predefined functions like int(), float(), str(), etc. Due to the user's casting (changing) of the objects' data types, this form of conversion is also known as **typecasting**.

What is Type Conversion in Python?

Suppose a situation where a person is traveling to America from India and that person wants to buy something from a store in America. But that person only has cash in INR(Indian Rupees). So the person goes to get their money converted from INR to USD (American Dollar) from a local bank in America.

The bank converts the INR into USD, and then the person can use the USD currency easily.

In the above scenario, there are two types of currencies, INR and USD, which you can consider as data types the bank has converted INR to USD, and this process is similar to **Type Conversion in Python!**

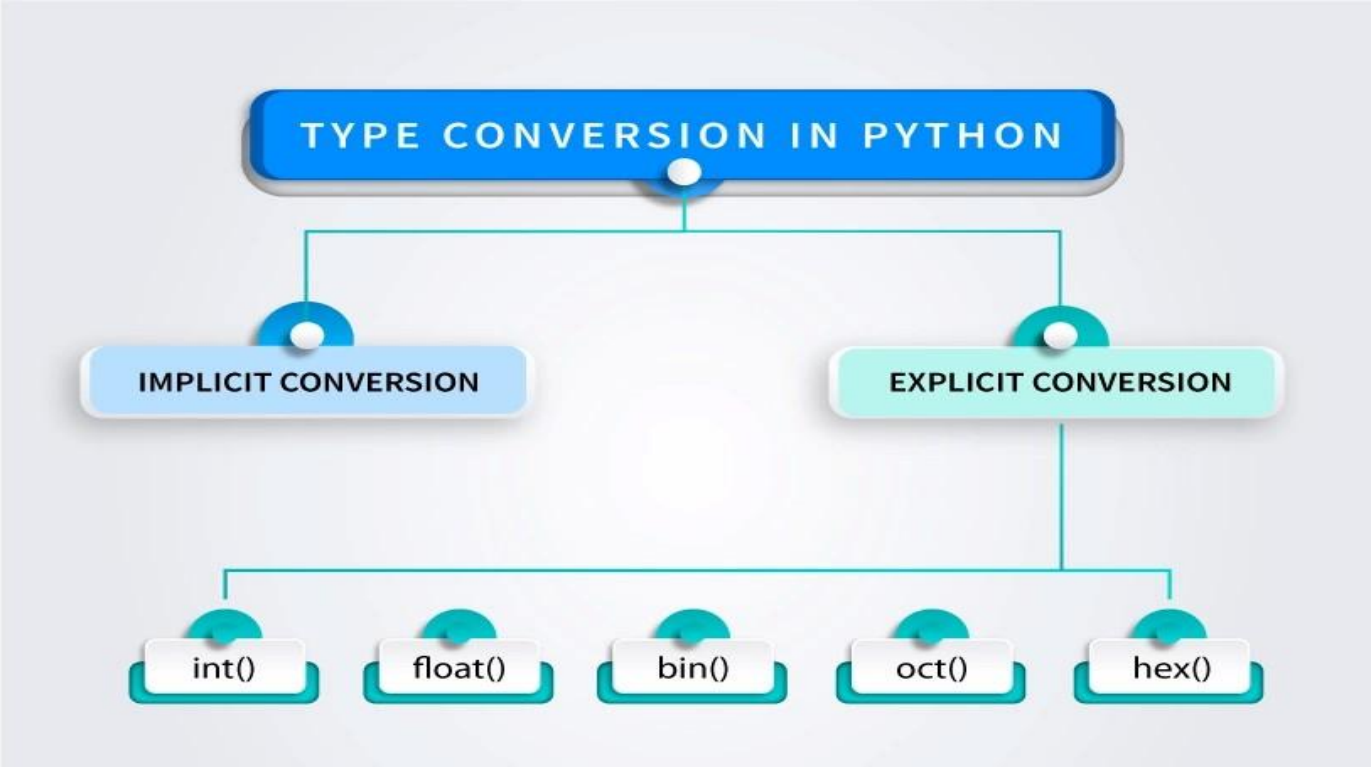
Now let's see the definition of type conversion in Python-

Type conversion is a process by which you can convert one data type into another. Python provides type conversion functions by which you can directly do so.

There are two types of type conversion in Python:

. **Implicit Type Conversion**

. Explicit Type Conversion



Explicit Type Conversion in Python

Explicit type conversion, also known as **Type Casting**, is when we manually convert one data type into another using built-in functions.

Examples of Explicit Conversion

Python provides several functions for explicit type conversion:

Function	Converts to
`int(x)`	Integer
`float(x)`	Floating-point number
`str(x)`	String
`list(x)`	List
`tuple(x)`	Tuple
`set(x)`	Set

Function	Converts to
`dict(x)`	Dictionary
`bool(x)`	Boolean

---

1. Converting String to Integer

```
num_str = "100"
num_int = int(num_str)  # Explicit conversion
print(num_int, type(num_int))  # Output: 100 <class 'int'>
```

🔻 If the string contains non-numeric characters (e.g., `"100abc"`), this conversion will fail with a `ValueError`.

---

2. Converting Float to Integer

```
num_float = 10.75
num_int = int(num_float)  # Decimal part is removed
print(num_int)  # Output: 10
```

🔻 **Note:** It truncates (removes) the decimal part instead of rounding.

---

3. Converting Integer to String

```
num = 50
num_str = str(num)
print(num_str, type(num_str))  # Output: "50" <class 'str'>
```

---

4. Converting List to Tuple

```
my_list = [1, 2, 3]
my_tuple = tuple(my_list)
print(my_tuple, type(my_tuple))  # Output: (1, 2, 3) <class 'tuple'>
```

---

5. Converting List to Set

```
my_list = [1, 2, 3, 3, 4]
my_set = set(my_list)
print(my_set, type(my_set))  # Output: {1, 2, 3, 4} <class 'set'>
```

🔻 **Note:** Duplicates are removed in the conversion.

---

6. Converting Boolean to Integer

```
print(int(True))   # Output: 1
```

```
print(int(False))  # Output: 0
```

🔻 `True` is converted to `1`, and `False` is converted to `0`.

---

7. Converting Dictionary Keys to a List

```
my_dict = {'a': 1, 'b': 2}
keys_list = list(my_dict.keys())
print(keys_list)  # Output: ['a', 'b']
```

Key Differences: Implicit vs. Explicit Conversion

Feature	Implicit Conversion	Explicit Conversion
Performed by Python	Yes	No (User does it)
Risk of Data Loss	No	Yes (e.g., `float → int`)
Requires Function Calls	No	Yes (e.g., `int()`, `str()`)